

**A
Project Report
On**

**MACHINE LEARNING BASED
TRANSFORMER INCIPIENT FAULT
DIAGNOSIS USING DISSOLVED GAS
ANALYSIS**

by

Jaladhi Shrivastava (2200970210041)

Parul Mittal (2200970210054)

Prikshit (2200970210060)

Arjun Dhaka (2200970210022)

Under the Guidance of

Dr. Md. Danish Equbal

Professor, Department of Electrical and Electronics Engineering



**DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING
GALGOTIAS COLLEGE OF ENGINEERING AND
TECHNOLOGY
GREATER NOIDA
(2022-2026)**

**A
Project Report
On
MACHINE LEARNING BASED
TRANSFORMER INCIPIENT FAULT
DIAGNOSIS USING DISSOLVED GAS
ANALYSIS**

**Submitted
In partial fulfilment of the requirements for the degree of
Bachelor of Technology
in
ELECTRICAL AND ELECTRONICS
ENGINEERING**

by
**Jaladhi Shrivastava (2200970210041)
Parul Mittal (2200970210054)
Priksht (2200970210060)
Arjun Dhaka (2200970210022)**

Under the Guidance of
Dr. Md. Danish Equbal
Professor, Department of Electrical and Electronics Engineering



**DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING
GALGOTIAS COLLEGE OF ENGINEERING AND
TECHNOLOGY
GREATER NOIDA
(MAY 2025-26)**

CERTIFICATE

This is to certify that the project entitled “**Machine Learning Based Transformer Incipient Fault Diagnosis using Dissolved Gas Analysis**” has been carried out by the team given below under my guidance in partial fulfilment of the degree of Bachelor of Technology in ELECTRICAL & ELECTRONICS ENGINEERING during the academic year 2025-2026 (8th Sem).

Jaladhi Shrivastava (2200970210041)

Parul Mittal (2200970210054)

Prikshit (2200970210060)

Arjun Dhaka (2200970210022)

Date:

Place: Greater Noida

Dr. Md. Danish Equbal

Project Guide

Dr. Kalpana Hazarika

Project Coordinator

Dr. Md. Danish Equbal

Prof. & HOD, EEE

External Examiner:

DECLARATION BY CANDIDATES

We hereby declare that the Project report entitled “**Machine Learning Based Transformer Incipient Fault Diagnosis using Dissolved Gas Analysis**” submitted by us to Galgotias College of Engineering and Technology in partial fulfilment of the requirement for the award of the degree of B. Tech in “Electrical & Electronics Engineering Department” is a record of Bonafide project work carried out by us under the guidance of Dr. Md. Danish Equbal. We further declare that the work reported in this project has not been submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Jaladhi Shrivastava (2200970210041)

Parul Mittal (2200970210054)

Prikshit (2200970210060)

Arjun Dhaka (2200970210022)

Date:

Place: Greater Noida

ACKNOWLEDGEMENT

I express my deep gratitude to all those who have provided their valuable support and guidance throughout the course of this project. First and foremost, I would like to thank Dr. Md. Danish Equbal, my project guide, for their insightful advice, constant encouragement, imparting knowledge and skills that have been crucial in the successful execution of this project.

I would also like to extend my sincere thanks to Dr. Md. Danish Equbal, Professor & HOD, EEE & Dr. Kalpana Hazarika, Project Coordinator, EEE, GCET, who have been instrumental in constructive feedback, which greatly contributed to the completion of this project.

Finally, I would like to thank my family for their continuous support, understanding, and patience during this project. Their unwavering belief in my abilities has been a constant source of motivation.

This project would not have been possible without the collective efforts and contributions of everyone mentioned above, and I am deeply thankful to all of them.

Jaladhi Shrivastava (2200970210041)

Parul Mittal (2200970210054)

Prikshit (2200970210060)

Arjun Dhaka (2200970210022)

PAGE INDEX

Topic	Page No.
TITLE PAGE	
CERTIFICATE	I
DECLARATION BY CANDIDATES	II
ACKNOWLEDGEMENTS	III
PAGE INDEX	IV
TABLE INDEX	VII
FIGURE INDEX	VIII
ABSTRACT	1
1. INTRODUCTION	
1.1 Background and Motivation	2
1.2 Problem Statement	3
1.3 Objectives	4
1.4 Scope of the Project	4
1.5 Report Organization	5
2. LITERATURE REVIEW	
2.1 DGA Techniques and Standards	6
2.2 IEC Ratio Method (Rogers Ratio)	7
2.3 Duval Triangle Method	7
2.4 Machine Learning in Transformer Diagnostics	8
2.5 Research Gap and Contribution	8
3. SYSTEM ARCHITECTURE	
3.1 Top-Level System Architecture	10
3.2 Transformer Under Study	11
3.3 Justification of Selection of Transformer	12

4.	METHODOLOGY	
4.1	Data Generation using MATLAB	14
4.2	Data Preprocessing	17
4.3	Feature Engineering	18
5.	MACHINE LEARNING MODELS	
5.1	Decision Tree	20
5.2	Random Forest	21
5.3	XGBoost	22
6.	TRAINING AND EVALUATION	
6.1	Overview of Training Pipeline	25
6.3	Data Preparation and Feature Construction	26
6.3	Data Splitting	26
6.4	Cross Validation Design	27
6.5	Model Training Procedure	27
6.6	Evaluation Metrics	28
7.	RESULTS AND ANALYSIS	
7.1	Model Performance Results	30
7.2	Feature Ablation Study	32
7.3	Classical Methods Comparison	33
7.4	Explainability using SHAP	35
7.5	Discussion of Results	35
8.	API AND UI INTEGRATION	
8.1	Backend API Design	37
8.2	User Interface Overview	38
9.	APPLICATIONS AND ADVANTAGES	
9.1	Applications	41
9.2	Advantages	42
10.	CONCLUSIONS AND FUTURE WORK	
10.1	Conclusions	43
10.2	Limitations	44

10.3	Future Scope	44
REFERENCES		46
APPENDICES		
A	MATLAB Code	48
B	Python Training Code	52
C	Benchmark Code	53
D	API Code	54

TABLE INDEX

Sr. No.	Title	Page No.
1	Transformer Specifications	12
2	Fault Class Summary and Gas Indicators	16
3	Feature Set Description	18
4	Model Hyperparameters	23
5	Cross-Validation F1 Scores	29
6	Fault-wise F1 Score Comparison	31
7	Feature Ablation Study Results	31
8	Classical Method vs ML Accuracy	33
9	API Input/Output Format	36

FIGURE INDEX

Sr. No.	Title	Page No.
1	System Architecture Block Diagram	10
2	3-Phase, Oil Filled Distribution Transformer	13
3	MATLAB Gas Dataset Generation Model Flow	14
4	Decision Tree Model	20
5	Random Forest Model	21
6	XGBoost Model	22
7	ML Pipeline Flow Diagram	24
8	Interactive UI Dashboard and Prediction Console	38-39

ABSTRACT

Power transformers are critical components of electrical distribution systems, and their unexpected failure can lead to major operational and economic losses. DGA (Dissolved Gas Analysis) is widely used to assess transformer health by analyzing gases generated due to electrical and thermal stresses in insulating oil. However, conventional interpretation methods such as the Rogers Ratio and Duval Triangle rely on fixed thresholds and often produce inaccurate or unclassified results.

This project presents a machine learning-based approach for transformer fault diagnosis using a physics-informed synthetic dataset generated in MATLAB. The dataset models a 100 kVA distribution transformer under seven fault conditions, including partial discharge, electrical faults, thermal faults, and normal operation. Three models—Random Forest, Decision Tree, and XGBoost—were trained and evaluated, each achieving approximately 97% accuracy.

A feature analysis showed that raw gas concentrations alone are sufficient for accurate classification, eliminating the need for traditional ratio-based features. SHAP (SHapley Additive exPlanations) analysis further confirmed that the model predictions align with known transformer fault mechanisms. The system is deployed using a FastAPI backend with a user interface, enabling real-time fault prediction and future integration with monitoring systems.

Keywords: *Transformer Fault Diagnosis, Dissolved Gas Analysis, Machine Learning, Random Forest, XGBoost, Decision Tree, FastAPI, Power Transformers*

CHAPTER-1

INTRODUCTION

1.1 Background of Power Transformers

Power transformers are really important for the alternating current power system. They help to change the voltage so that power can be sent over distances and used by people in their homes and businesses. For example one small power transformer can supply power to people. If this transformer breaks down it can cause a lot of problems. People may not have power for a time hospitals and water systems may not work and businesses can lose money.

Power transformers use oil to keep them cool and to prevent electricity from escaping. This oil is very stable when everything is working normally.. When there are problems with the electricity like sparks or too much heat the oil can start to break down. This breakdown can create gases inside the oil. By looking at these gases, we can tell if there is a problem with the power transformer. This is called Dissolved Gas Analysis or DGA for short.

DGA is an useful tool for keeping power transformers working properly. It has been standardized by organizations like IEC and IEEE.. Power transformers are all different and the oil inside them is different too. So we need ways to use DGA to find problems. We need to use computers and data to help us understand what is going on with the power transformers. Power transformers and DGA are very important, for making sure we have power when we need it.

1.2 Problem Statement

The traditional techniques for DGA interpretation such as the Rogers Ratio Method, Duval Triangle and Key Gas Method are based on predetermined threshold values and empirical principles based on historical observations. These techniques have been widely used for transformer fault diagnosis . However, these techniques have several limitations when used in modern power systems:

- Traditional methods sometimes fail to adequately identify complicated or many simultaneous issues. For example, the IEC / Rogers Ratio approach can give an inconsistent or incorrect diagnosis when overlapping fault situations exist.
- If the values of gas concentrations do not fall inside the boundaries of the pre-defined regions or classes, techniques such as the Duval Triangle can be inconclusive.
- Traditional rule-based systems cannot learn from newly available data, adapt to changing operating conditions, contemporary transformer technology, or changing fault patterns.
- Manual interpretation of DGA findings based on fixed rules is wasteful and time consuming for large-scale power systems that require continuous, real-time condition monitoring and fast fault evaluation.

These limitations are due to the unavailability of an efficient, automated and highly accurate fault diagnosis system for current power transformers. As DGA is essential for transformer status monitoring and preventative maintenance, the demand for intelligent and data-driven methods that can offer dependable and adaptive fault identification in modern power systems is increasing.

1.3 Objectives

The main goal of this project is to create an reliable system to detect faults in transformers using machine learning and data from Dissolved Gas Analysis.

The specific goals are:

- Simulate a physics-informed, balanced, labelled DGA dataset using a MATLAB simulation model grounded in IEC-aligned transformer faults for a 100 kVA, 11 kV/415 V, ONAN distribution transformer.
- Implement and evaluate three machine learning models — Random Forest, Decision Tree, and XGBoost — for seven-class transformer fault classification using cross-validation and standardised metrics.
- Implement and benchmark classical DGA interpretation methods (IEC Ratio and Duval Triangle) against the ML models on identical test data, with per-class performance breakdown.
- Deploy the trained models via a FastAPI backend integrated with a UI dashboard that accepts raw gas concentration inputs and returns real-time fault predictions with API-key authentication.

1.4 Scope of the Project

This project focuses on developing and evaluating a machine learning-based fault detection system for power transformers using Dissolved Gas Analysis (DGA) data. The scope of work encompasses:

- Development and acquisition of datasets for major fault-indicator gases such as hydrogen and methane.
- Review of existing types of transformer faults including electrical discharge, thermal degradation and normal operation.
- Research and assess a variety of diagnostic procedures to establish the optimal approach.

- Analyse the performance of the system based on the quantitative assessment measures, such as classification accuracy and fault detection coverage.

The project does not include building the system into hardware or using it in the field now but it is designed to be used in those ways later.

1.5 Report Organization

This report is systematically organized to present the development and evaluation of a machine learning-based transformer fault detection framework. Chapter 1 introduces the background, problem statement, objectives, scope, and overall structure of the work. Chapter 2 reviews existing literature, covering conventional Dissolved Gas Analysis (DGA) techniques and recent advancements in machine learning-based diagnostics, followed by identification of the research gap. Chapter 3 describes the overall system architecture, including the transformer under study and its operational parameters. Chapter 4 details the methodology, encompassing dataset generation using MATLAB, preprocessing steps, and feature engineering strategies. Chapter 5 presents the machine learning models employed, while Chapter 6 outlines the training procedures and evaluation metrics used to assess performance. Chapter 7 consolidates the results and analysis, including model performance, feature ablation study, comparison with classical methods, and explainability using SHAP. Chapter 8 discusses the implementation aspects through API and user interface integration. Chapter 9 highlights practical applications and advantages of the proposed system. Finally, Chapter 10 concludes the report by summarizing key findings, discussing limitations, and suggesting directions for future work, followed by references and appendices containing implementation details and code.

CHAPTER -2

LITERATURE REVIEW

2.1 DGA Techniques and Standards

The foundational work on Dissolved Gas Analysis dates to the 1960s and 1970s, when researchers at Westinghouse and CEGB identified correlations between specific gases and specific fault mechanisms in oil-filled equipment. Rogers formalized this into the ratio-based approach that became codified in both IEEE C57.104 and IEC 60599 [1]. The key insight was that rather than absolute gas concentrations, which vary enormously with transformer size, oil volume, and loading history, the ratios between gases exhibit more stable fault-class correlations.

Duval subsequently proposed the triangular graphical method as a complement to the ratio approach, using the relative percentages of acetylene, ethylene, and methane within a ternary coordinate system to define fault zones [2]. The Duval Triangle has the advantage of being free from zone boundaries between adjacent faults (the triangle covers the entire composition space), but it excludes CO and CO₂ information and has no designated Normal region.

IEC 60599:2022 represents the current international standard for DGA interpretation, incorporating both classical ratio methods and guidance on supplemental analysis using CO/CO₂ ratios for paper insulation degradation assessment [3]. IEEE C57.104-2019 similarly covers concentration limits and interpretation guidelines for North American practice [4].

2.2 IEC Ratio Method (Rogers Ratio)

The Rogers Ratio method classifies transformer faults using three gas ratios: $R1 = CH_4/H_2$, $R2 = C_2H_2/C_2H_4$, and $R3 = C_2H_4/C_2H_6$. Each ratio is assigned a code (0, 1, or 2) based on threshold ranges, and the combination of three codes maps to a fault type. The method was validated on early transformer fleet data and remains in active use in utility maintenance programs worldwide.

However, its practical limitation, particularly for automated systems, is the high rate of cases that fall outside the defined code combinations. In such cases, the method returns no classification, requiring either manual expert intervention or fallback to an alternative method. Extensive field studies have reported that between 20-40% of real transformer DGA samples fall into undefined zones depending on the asset population and loading conditions.

2.3 Duval Triangle Method

Duval's method has been refined into multiple triangle variants (Triangle 1 for general use, Triangles 2–5 for specific conditions), with the original Triangle 1 mapping the CH_4 – C_2H_4 – C_2H_2 ternary composition space into six fault zones: PD, T1, T2, T3, D1, and D2 [2]. Its strength is that every point in the triangle maps to some zone, eliminating the UNKNOWN category for samples with non-zero values of the three constituent gases.

The principal weaknesses are the exclusion of Normal condition, the exclusion of H_2 , CO, and CO_2 from the analysis, and the sensitivity to the relative contribution of acetylene, which can be negligible in thermal faults and thus push points into the T1 zone regardless of the actual thermal severity.

2.4 Machine Learning in Transformer Diagnostics

Several research groups have applied supervised machine learning to DGA-based fault classification over the past two decades. Senoussaoui et al. demonstrated Support Vector Machine (SVM) classification on IEC standard datasets, reporting accuracies of 85-90% with Radial Basis Function kernels on relatively small (200–500 sample) datasets. The limitation of SVMs in this domain is their sensitivity to class imbalance, which is a persistent issue with real field data, where Normal and early-stage faults are underrepresented [5].

Random forest methods were applied by Wang et al. to DGA fault classification, achieving 93-95% accuracy on multi-source aggregated datasets. Their work identified feature importance patterns consistent with IEC physics: C_2H_2 as the dominant arc indicator, H_2 as the primary PD marker, and C_2H_4 as the key high-temperature thermal feature. These findings are consistent with our own SHAP analysis results [6].

More recently, deep learning approaches, particularly 1D convolutional neural networks and LSTM variants, have been explored for DGA sequence analysis where temporal gas evolution data is available [7]. While these methods show promise for trend-based diagnosis, they require historical time-series data from continuous monitoring, which is not available for the single-sample snapshot DGA paradigm that remains standard in most utility maintenance programs.

2.5 Research Gap and Contribution

The contribution of this project relative to the existing literature lies in three areas: (a) controlled dataset generation using a validated physics-informed model that avoids the dataset heterogeneity problems; (b) a rigorous feature ablation study comparing raw gas features against engineered ratios, which

has not been reported systematically in prior work; and (c) deployment of a production-ready API backend with UI integration and SHAP-based explainability — moving beyond academic benchmarking to operational implementation.

CHAPTER-3

SYSTEM ARCHITECTURE

3.1 Top-Level System Architecture

The block diagram representation of the system architecture is given in Fig.3.1. The complete system follows a modular layered architecture designed for maintainability and production deployment. The data flow proceeds from the transformer through the dataset, preprocessing, model inference, API layer, and finally to the UI dashboard.

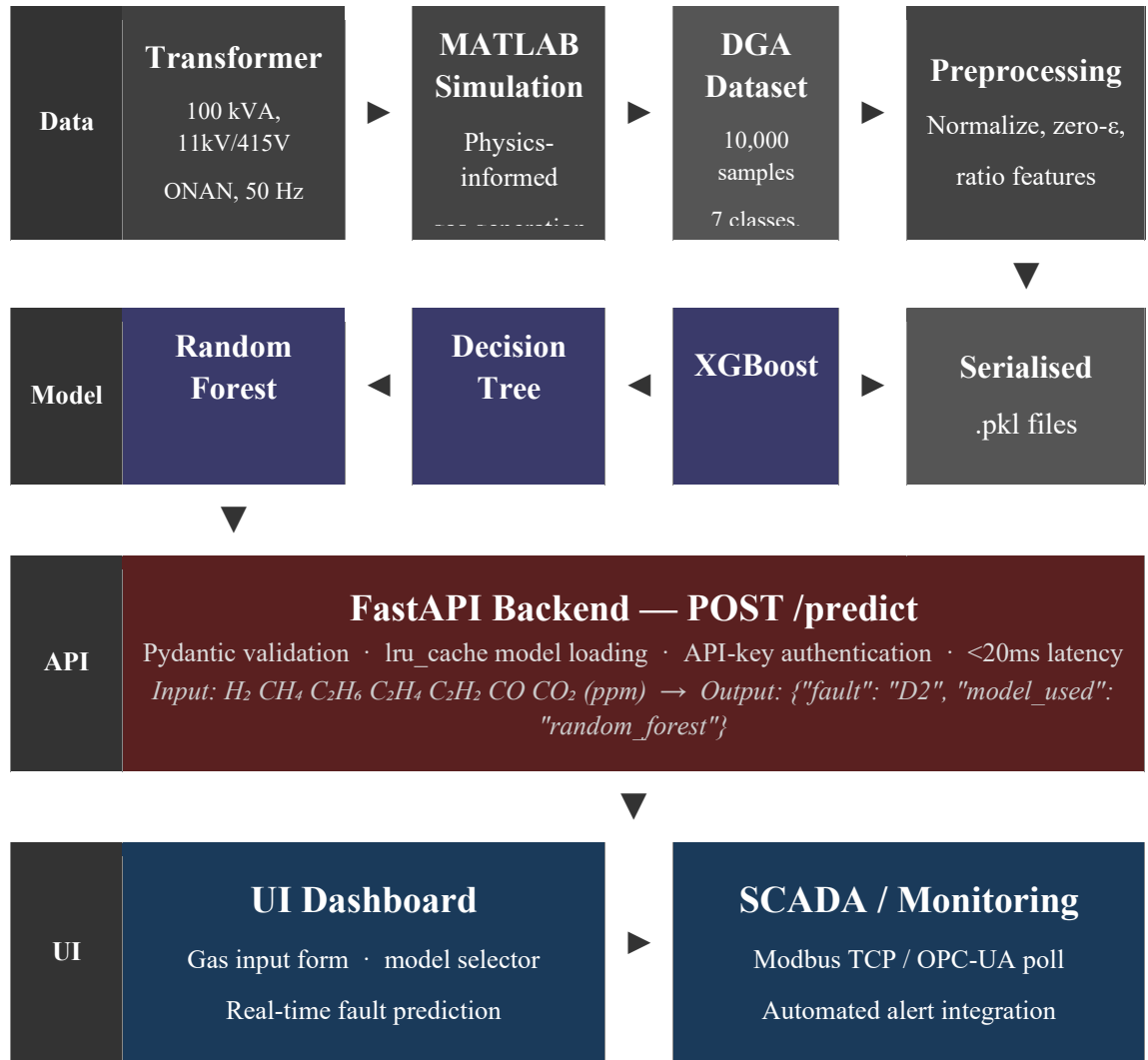


Figure 3.1: System Architecture Block Diagram

The architecture is organized into four logical layers:

- **Data Layer:** The MATLAB-generated CSV dataset (`transformer_dga_dataset.csv`) serves as the training data source. In production, this would be replaced by a live DGA instrument feed or SCADA data connector. The preprocessing pipeline (`src/preprocessing.py`) handles all data cleaning, normalization, and feature engineering in a single function call.
- **Model Layer:** Three serialized model files (`random_forest.pkl`, `decision_tree.pkl`, `xgboost.pkl`) stored in the `models/` directory. Models are loaded using `joblib` with `lru_cache` to ensure each model is deserialized from disk only once per server process lifetime, keeping inference latency below 20 ms per request.
- **API Layer:** A FastAPI application (`api/app.py`) exposes the `/predict` POST endpoint. Pydantic models enforce input type validation and non-negativity constraints on gas values. The endpoint accepts a model selection parameter, defaulting to Random Forest. API key authentication is implemented for the production UI integration.
- **UI Layer:** A frontend dashboard (developed separately) communicates with the API over HTTP. The UI allows operators to manually enter seven gas concentration values (H_2 , CH_4 , C_2H_6 , C_2H_4 , C_2H_2 , CO , CO_2 in ppm), select the ML model, and receive an instantaneous fault classification with associated description and recommended maintenance action.

3.2 Transformer Under Study

The system is designed around a three-phase, oil-filled distribution transformer, shown in Fig. 3.2, which is representative of the most widely deployed class of equipment in Indian distribution networks. The specific configuration was selected to ensure the simulation model, fault physics,

and dataset reflect realistic operating conditions rather than laboratory-scale or transmission-class equipment.

Table 3.1: Transformer Specifications

Parameter	Value
Type	Three-phase oil-filled distribution transformer
Rated Power	100 kVA
Primary Voltage	11 kV (line-to-line)
Secondary Voltage	415 V (line-to-line)
Frequency	50 Hz
Cooling Method	ONAN (Oil Natural, Air Natural)
Rated Current (LV Side)	≈ 139 A
Temperature Rise (Rated)	65°C above ambient
Insulating Medium	Mineral oil (IEC 60296)
Connection	Delta-Star (Dyn11)

3.3 Justification of Selection of Transformer

The 100 kVA, 11 kV/415 V configuration was chosen for several converging reasons. First, this rating constitutes the backbone of rural and semi-urban distribution networks across India, making it the most numerically significant transformer category in utility asset registers. DGS distribution utilities in several Indian states report that 60–70% of their transformer population falls in the 25–250 kVA range at 11 kV primary voltage.

Second, ONAN cooling is the simplest and most common cooling configuration. This implies that oil temperature is governed primarily by load current and ambient temperature, without the confounding effect of

forced cooling control. This makes the physics model tractable and the temperature-to-gas-generation mapping reliable.

Third, the 65°C rated temperature rise corresponds directly to the standard temperature boundaries used in IEC 60599 fault classification: T1 (below 300°C), T2 (300–700°C), and T3 (above 700°C) represent extreme overloading or hotspot conditions relative to this baseline, and the gas generation model can be meaningfully parameterized around these boundaries.

Fourth, using a single, well-characterized transformer type avoids the dataset heterogeneity, where samples from different transformer sizes, oil types, and operating histories are mixed. This introduces systematic noise that depresses classification performance and complicates comparison.

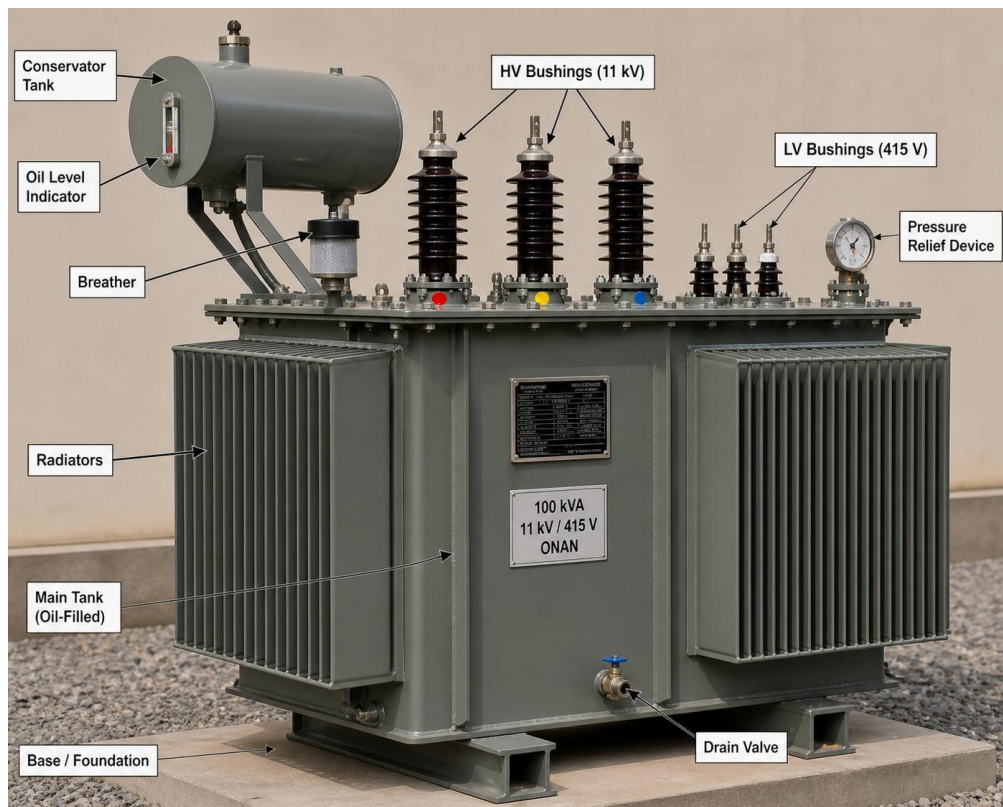


Figure 3.2: 3-Phase, Oil Filled Distribution Transformer

CHAPTER-4

METHODOLOGY

4.1 Dataset Generation using MATLAB

This project generated a synthetic dataset using a physics-informed MATLAB simulation rather than relying on externally aggregated field datasets, which carry the inherent problems of mixed transformer ratings, unknown oil condition histories, inconsistent labelling conventions across different IEC/IEEE editions, and severe class imbalance favouring Normal operation. A block diagram depicting the model flow of gas dataset generation is given in Fig.4.1 below.

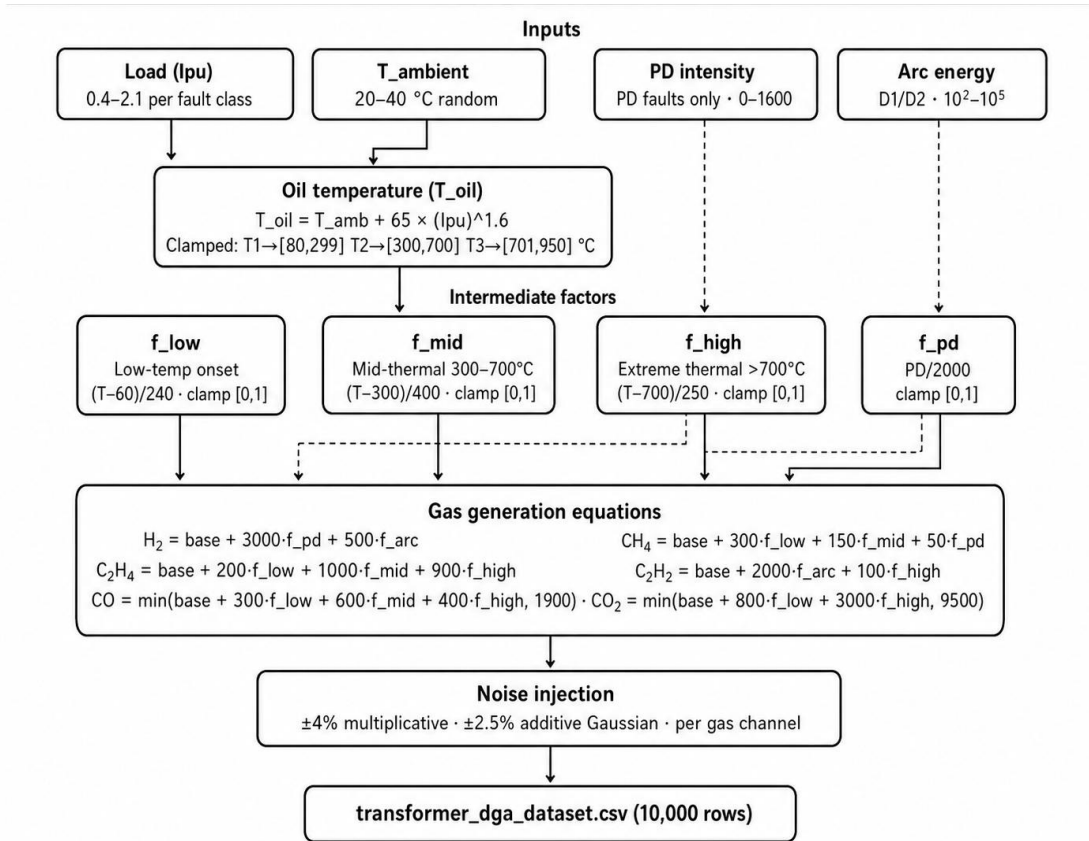


Figure 4.1: MATLAB Gas Dataset Generation Model Flow

The simulation is grounded in the following causal chain, which reflects the actual fault physics of oil-filled transformers:

**Electrical / Thermal Stress → Oil Temperature (T_{oil}) → Gas
Generation Rates (H₂, CH₄, C₂H₆, C₂H₄, C₂H₂, CO, CO₂)**

Each of the seven fault classes is assigned a characteristic load per-unit (lpu) range and stress parameters. The oil temperature is computed from the rated temperature rise model:

$$T_{oil} = T_{ambient} + \Delta T_{rated} \times (lpu)^{1.6}$$

where,

- T_{oil}= Top oil temperature of the transformer
- T_{ambient}= Ambient surrounding temperature
- ΔT_{rated}= Rated top oil temperature rise above ambient temperature
- l_{pu}= Per-unit load on the transformer
- 1.6= Empirical exponent representing nonlinear thermal behavior of transformer oil heating

For discrete fault types, oil temperature is clamped to the IEC-specified thermal bands- T1 is restricted to [80°C, 299°C], T2 to [300°C, 700°C], and T3 to [701°C, 950°C].

The gas generation function maps oil temperature, partial discharge intensity (PD), and electrical arc energy (EA) to seven gas concentrations using physics-based transfer functions. The key relationships encoded are:

- Hydrogen (H₂): Elevated by both PD (electron avalanche ionisation) and thermal cracking above 300°C. The PD contribution is modelled as $3000 \times f_{pd}$ where $f_{pd} = \min(1, PD/2000)$, reflecting the empirically observed dominance of H₂ in partial discharge conditions.
- Methane (CH₄): Primarily a low-temperature thermal product (300–450°C range), produced by methane dehydrogenation of oil under moderate thermal stress.
- Ethane (C₂H₆): A low-to-moderate temperature indicator, produced alongside CH₄ with a suppression factor at higher temperatures where dehydrogenation favours ethylene.

- Ethylene (C_2H_4): The key indicator of sustained high-temperature thermal stress (above $300^\circ C$). Production peaks in the T2 and T3 ranges ($300-700^\circ C$, $>700^\circ C$), making it the primary discriminator of severe thermal faults.
- Acetylene (C_2H_2): Produced almost exclusively under arc discharge conditions where localised plasma temperatures exceed $1000^\circ C$. The model assigns $2000 \times f_{arc} + 100 \times f_{high}$, where $f_{arc} = \min(1, \log_{10}(EA+1)/5)$. This reflects the experimentally observed non-linear relationship between arc energy and acetylene production.
- Carbon Monoxide (CO): Generated by cellulose (paper insulation) pyrolysis, increasing steadily with oil temperature. Capped at 1,900 ppm per practical instrument range limits.
- Carbon Dioxide (CO_2): A secondary cellulose decomposition product, with the ratio CO_2/CO often used as an indicator of insulation paper condition. Capped at 9,500 ppm.

After generating nominal gas values, Gaussian multiplicative noise (4% standard deviation) and additive noise (2.5% of gas value) are injected to simulate the realistic measurement uncertainty of field DGA instruments. This noise level is consistent with the $\pm 5-10\%$ reproducibility reported for standard gas chromatography methods used in DGA laboratories. Table 4.1 shows the classification of transformer fault conditions along with their corresponding labels, per-unit loading (l_{pu}) ranges, thermal operating bands, and key stress parameters. The table categorizes different electrical and thermal faults such as Partial Discharge (PD), Spark Discharge (D1), Arc Discharge (D2), and thermal faults (T1, T2, and T3) based on transformer loading and temperature severity. It also highlights the dominant stress mechanisms responsible for gas generation under each operating condition.

Table 4.1: Fault Class Summary and Gas Indicators

Fault Class	Label	lpu Range	Thermal Band	Key Stress Parameter
Normal	NORMAL	0.4–1.0	Ambient	None
Partial Discharge	PD	0.5–1.0	Low	PD = 100–1600
Spark Discharge	D1	0.6–1.2	Low–moderate	PD + EA (low)
Arc Discharge	D2	0.7–1.5	High (arc)	EA = 10^2 – 10^5
Thermal <300°C	T1	0.9–1.2	80–299°C	Overload
Thermal 300–700°C	T2	1.2–1.6	300–700°C	Severe overload
Thermal >700°C	T3	1.6–2.1	701–950°C	Extreme overload

The final dataset contains 10,000 samples, balanced at approximately 1,428–1,429 samples per class, ensuring that no single fault class dominates the training process.

4.2 Data Preprocessing

The preprocessing pipeline is implemented in `src/preprocessing.py` and executes the following steps in sequence:

Label Normalisation: The `Fault_Class` string column is stripped and uppercased, then mapped to integers via a fixed `LABEL_MAP` dictionary: PD→0, D1→1, D2→2, T1→3, T2→4, T3→5, NORMAL→6. Any

unmapped label triggers a `ValueError` with diagnostic output of the offending values.

Zero Handling: Gas concentrations of exactly zero are replaced by $\varepsilon = 10^{-6}$ before ratio computation to prevent division by zero. Critically, this replacement is applied only to the seven gas concentration columns — the integer label column is encoded after epsilon replacement to prevent label corruption (a bug that manifests as a float y-vector and causes sklearn to throw 'Unknown label type: continuous').

Feature Engineering: Five ratio features are computed: CH_4/H_2 , $\text{C}_2\text{H}_6/\text{CH}_4$, $\text{C}_2\text{H}_2/\text{C}_2\text{H}_4$, $\text{C}_2\text{H}_4/\text{C}_2\text{H}_6$, CO_2/CO . These are appended to the dataframe to give a 12-feature matrix (7 raw + 5 ratios), though the feature ablation study subsequently showed that 7 raw features alone are sufficient.

4.3 Feature Engineering and Ablation

The five ratio features were engineered following the conventions of the Rogers Ratio and Duval Triangle methods. The intent was to provide the ML models with domain-informed derived features that compress the diagnostic knowledge encoded in classical methods. However, this assumption was tested rigorously through an ablation study (detailed in Chapter 7).

Table 4.2: Feature Set Description

Feature	Fault Sensitivity
CH ₄ /H ₂	Discriminates PD (low) from thermal faults (high)
C ₂ H ₆ /CH ₄	Thermal progression indicator (T1 vs T2)
C ₂ H ₂ /C ₂ H ₄	Arc indicator — high for D1/D2, near-zero for thermal
C ₂ H ₄ /C ₂ H ₆	High-temperature thermal indicator (T2, T3)
CO ₂ /CO	Paper insulation degradation severity

CHAPTER-5

MACHINE LEARNING MODELS

Three tree-based ensemble models were selected for this classification task. The selection criteria were: native handling of mixed-scale features without normalisation, robustness to moderate feature redundancy, interpretability through feature importance, compatibility with SHAP explanations, and established performance on structured tabular data.

5.1 Decision Tree

The Decision Tree serves as an interpretable baseline within the tree family. Unlike Random Forest, a single decision tree partitions the feature space through a sequence of binary splits optimized by the Gini impurity criterion as shown in Fig.5.1. Its output is a human-readable tree structure where each node encodes a threshold on one feature, and each leaf assigns a class label. This explicit structure allows the prediction logic to be fully traced, making it valuable for viva demonstration and regulatory contexts where model transparency is required. The decision tree was trained with stratified splitting, the same train-test partition as the other models, and default hyperparameters for maximum depth and minimum samples per leaf. The fully-grown tree achieves comparable accuracy to the ensemble methods on this dataset, suggesting that the fault boundaries in the feature space are predominantly axis-aligned, a characteristic of tree-based models.

An additional advantage of the Decision Tree in this application is its ability to directly reflect domain-relevant thresholds in gas concentrations. The learned splits often correspond to physically meaningful boundaries (e.g., elevated acetylene for arcing faults or increased ethylene for thermal faults), which reinforces confidence in the model’s alignment with transformer fault physics. However, the absence of ensemble averaging makes it more

sensitive to small variations in the dataset, meaning that while it performs strongly here, its generalisation may degrade slightly under noisier real-world conditions compared to ensemble methods.

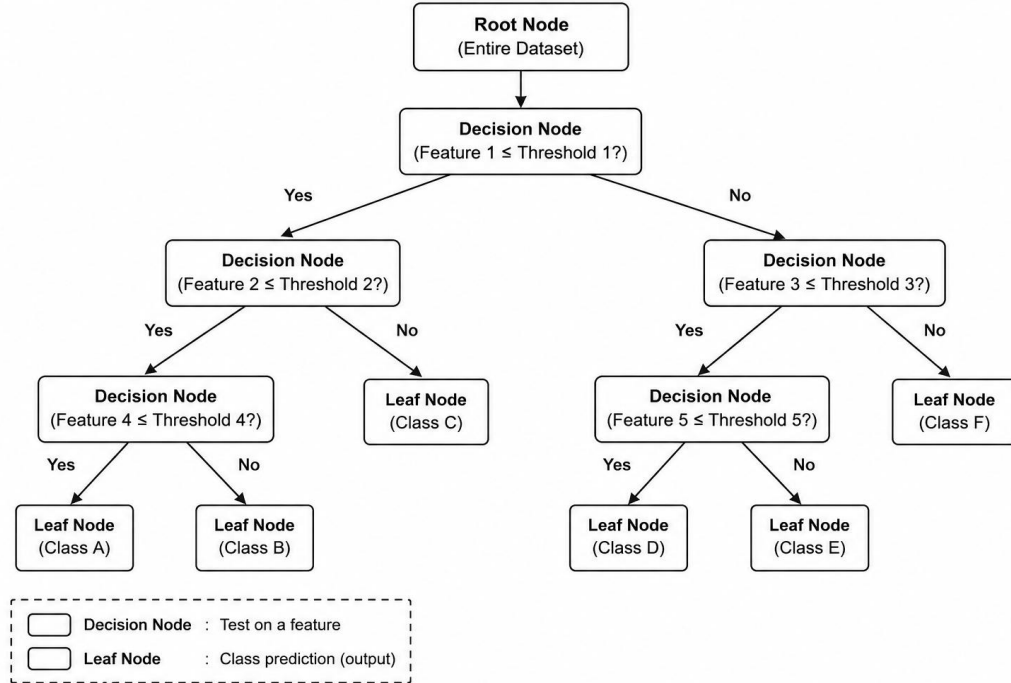


Figure 5.1: Decision Tress Model

5.1 Random Forest

Random Forest builds an ensemble of decision trees where each tree is trained on a bootstrap sample of the training data (bagging) and evaluates only a random subset of features at each split (feature subsampling). The final prediction is the majority vote across all trees, as shown in Fig.5.2. These two randomization mechanisms are independently effective at reducing variance: bagging addresses the high variance of individual decision trees, and feature subsampling decorrelates the individual trees in the ensemble.

For this project, the Random Forest was configured with 300 estimators, `class_weight='balanced'` (which assigns higher loss weight to minority classes proportional to their inverse frequency — though the dataset is

balanced, this provides robustness if future real-world data is imbalanced), and `random_state=42` for reproducibility. The `max_features` parameter defaults to 'sqrt' ($\sqrt{12} \approx 3.5$, rounded to 3 features per split), which is the standard empirically recommended value for classification tasks.

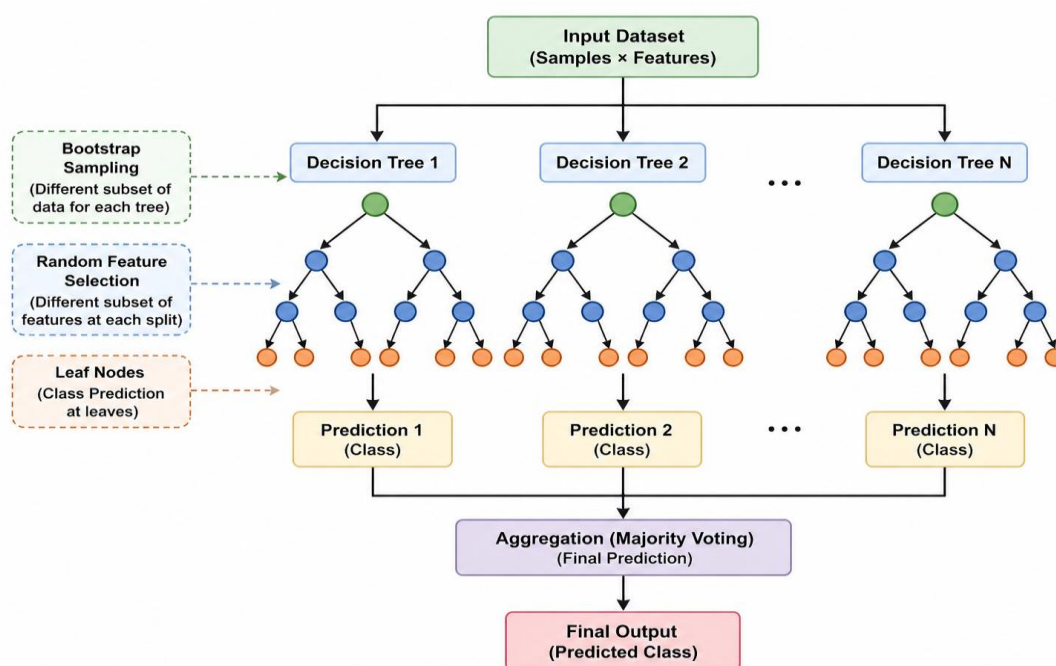


Figure 5.2: Random Forest Model

5.3 XGBoost

XGBoost (Extreme Gradient Boosting) is a gradient boosting framework that builds trees sequentially, where each new tree is optimised to correct the residual errors of the current ensemble. Unlike Random Forest, where trees are independent, XGBoost trees are trained in a stage-wise manner guided by the gradient of the loss function. Regularisation terms (L1 on leaf weights, L2 on leaf scores) are incorporated directly into the objective function, providing a principled mechanism for controlling overfitting. It is shown in Fig.5.3.

The XGBoost configuration used in this project: `n_estimators=500`, `learning_rate=0.03`, `max_depth=6`, `subsample=0.8`, `colsample_bytree=0.8`, `gamma=0.2`. The relatively low learning rate combined with high `n_estimators` allows fine-grained gradient descent convergence while the subsampling parameters provide stochastic regularisation.

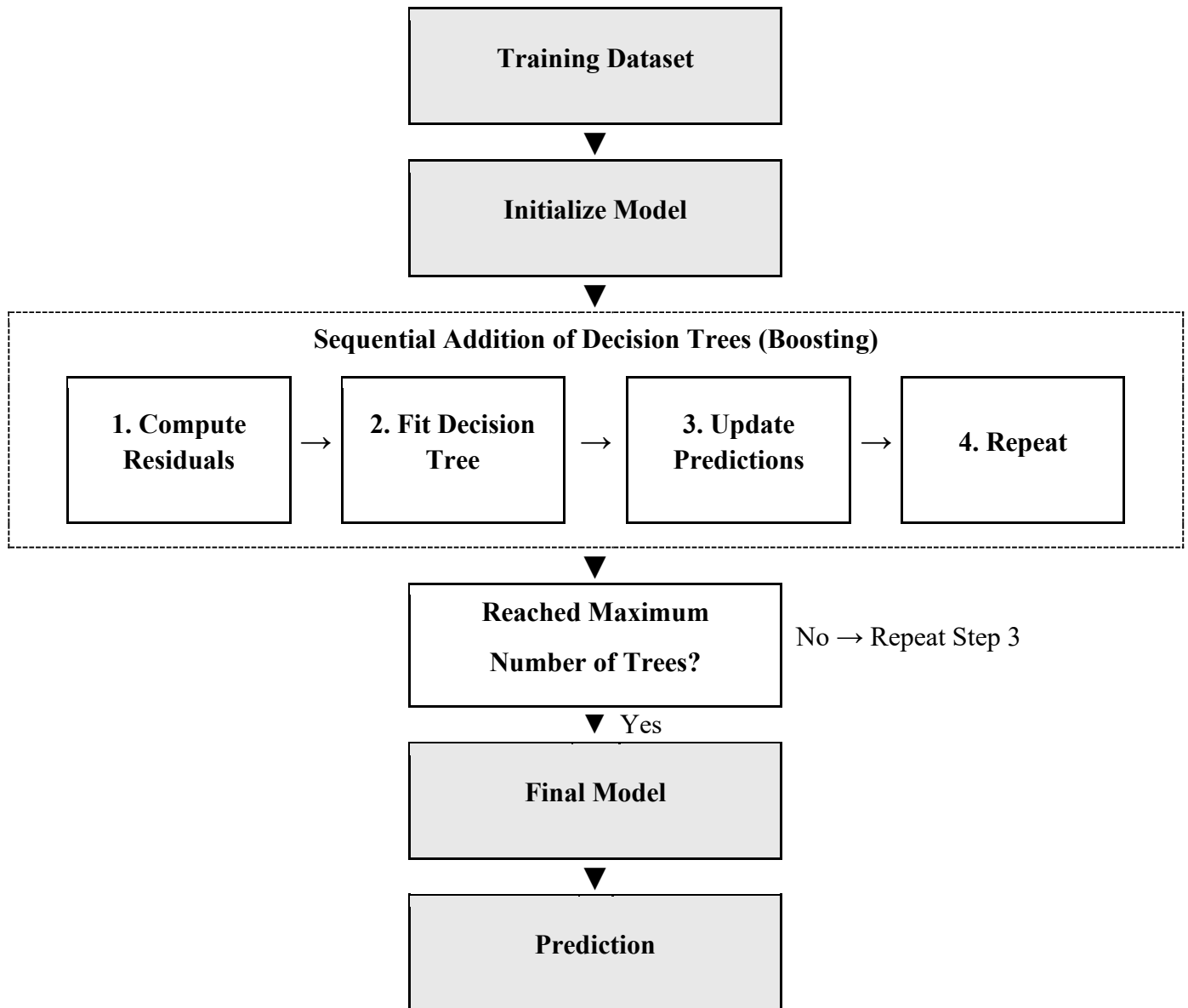


Figure 5.3: XGBoost Model

All three models were evaluated on an identical train-test split (80:20, stratified by fault class, `random_state=42`) and five-fold stratified cross-validation on the training set. The StratifiedKFold strategy ensures that each

fold contains approximately equal class proportions, providing unbiased F1 estimates.

Table 5.1: Model Hyperparameters

Model	Key Hyperparameters	CV F1-macro	Std Dev
Random Forest	n_estimators=300, class_weight=balanced, max_features=sqrt	0.9620	±0.0047
Decision Tree	criterion=gini, default depth, stratified split	0.9587	±0.0077
XGBoost	n_est=500, lr=0.03, max_depth=6, subsample=0.8	0.9598	±0.0043

CHAPTER-6

TRAINING AND EVALUATION

6.1 Overview of Training Pipeline

The training and evaluation framework follows a structured machine learning pipeline, as illustrated in Fig.6.1. The process begins with dataset ingestion from CSV format, followed by preprocessing and feature engineering to construct the final input representation. The dataset is then partitioned into training and testing subsets, after which multiple machine learning models are trained using cross-validation.

The trained models are subsequently evaluated on a strictly held-out test set to ensure unbiased performance estimation. Finally, the selected model is serialized and integrated into an API layer for real-time fault prediction.

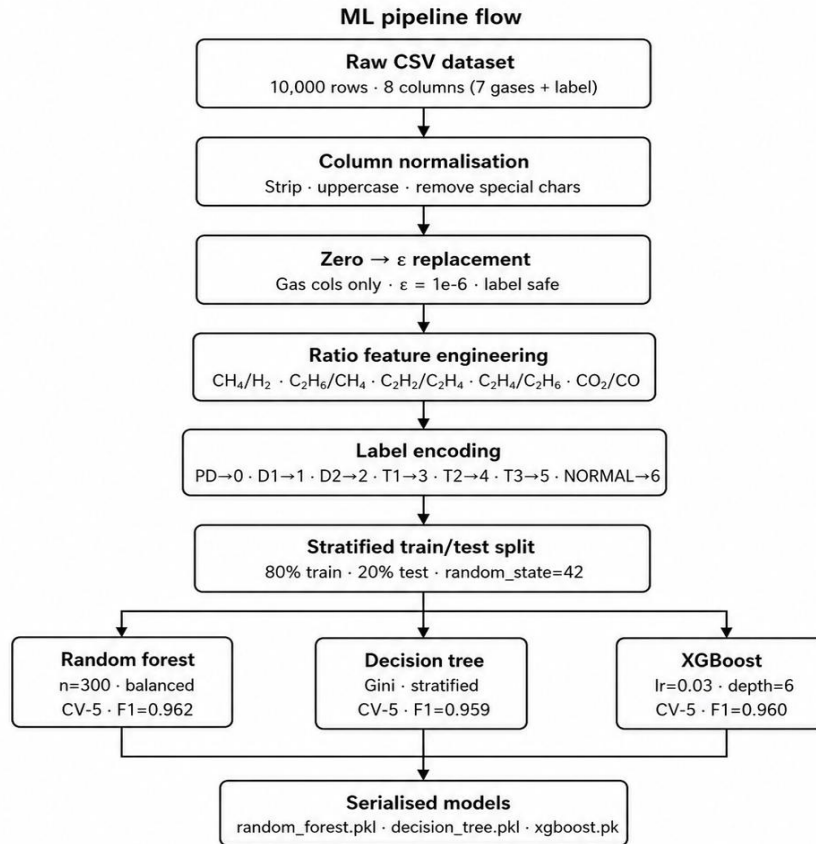


Figure 6.1: Machine Learning Pipeline Flow Diagram

This pipeline ensures a clear separation between data preparation, model development, validation, and deployment stages, thereby maintaining methodological rigor and reproducibility.

6.2 Data Preparation and Feature Construction

The input dataset consists of 10,000 samples representing transformer operating conditions through Dissolved Gas Analysis (DGA). Each sample is converted into a structured feature vector containing gas concentration values and derived features as defined in the feature engineering stage.

Before model training, the dataset undergoes preprocessing to ensure consistency and suitability for learning algorithms. This includes validation of feature ranges, handling of numerical stability, and preparation of the final feature matrix X and label vector y .

The processed dataset serves as the input to the classification pipeline where gas inputs are mapped to fault categories through the trained machine learning model.

6.3 Data Splitting

To evaluate generalisation performance, the dataset is divided into training and testing subsets using stratified random sampling:

- Training set: 8,000 samples (80%)
- Test set: 2,000 samples (20%)

The split is implemented using:

`train_test_split(test_size=0.2, random_state=42, stratify=y)`

Stratification ensures that all seven fault classes (PD, D1, D2, T1, T2, T3, NORMAL) are proportionally represented in both subsets. This results in approximately:

- ~286 samples per class in the test set
- ~1,142-1,143 samples per class in the training set

The test set is strictly held out and is not used during model training, hyperparameter tuning, or feature selection. This guarantees that final performance metrics reflect true out-of-sample behaviour.

6.4 Cross Validation Design

Model training is performed using stratified five-fold cross-validation to ensure robustness and stability of results. The configuration used is:

StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

In each fold:

- 80% of training data (6,400 samples) is used for model fitting
- 20% (1,600 samples) is used for validation

This process is repeated across all five folds, ensuring that each sample is used for validation exactly once.

The final cross-validation performance is reported as:

- Mean macro F1-score across folds
- Standard deviation, indicating model stability

Cross-validation plays a critical role in:

- Preventing overfitting
- Guiding model selection
- Validating generalisation prior to test evaluation

6.5 Model Training Procedure

Following data preparation and splitting, machine learning models are trained using the cross-validation framework described above. Each model learns the mapping:

Feature Vector (Gas Data) → Fault Class

The training process aligns with the classification flow illustrated in Figure 2, where:

- Input gas concentrations are transformed into feature vectors
- The trained classifier assigns one of the seven fault categories
- The prediction can be further used for diagnostic interpretation or recommended action

Model training is conducted exclusively on the training set, with hyperparameters implicitly validated through cross-validation performance.

6.6 Evaluation Metrics

Model performance is evaluated on the held-out test set using multiple complementary metrics:

1. **Accuracy:** The proportion of correctly classified samples across all 2,000 test instances. While intuitive, accuracy alone may not reflect per-class performance.
2. **Precision (Per Class):** Measures the reliability of positive predictions for each fault class

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. **Recall (Per Class):** Measures the model's ability to correctly identify all instances of a given fault class.

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. **F1-Score (Per Class):** The harmonic mean of precision and recall, providing a balanced measure of classification performance.
5. **Macro-Averaged F1 Score:** The unweighted average of F1 scores across all classes. This is the primary evaluation metric, as it treats all fault types equally regardless of frequency.

6. Confusion Matrix: A 7×7 matrix capturing the complete classification behaviour across all fault classes. This provides detailed insight into:

- Misclassification patterns
- Class-specific weaknesses
- Overlap between fault signatures

CHAPTER-7

RESULTS AND ANALYSIS

7.1 Model Performance Summary

All three models achieved 97% test accuracy on the 2,000-sample test set, with macro-averaged precision, recall, and F1 all at 0.97. This consistent performance across three architecturally distinct models, bagging ensemble, single tree, and sequential boosting, as shown in Table 7.1. This suggests that the fault classification boundaries in this feature space are well-defined and learnable by any sufficiently expressive tree-based model.

Table 7.1: Cross Validation F1 Scores

Fault Class	RF Precision	RF Recall	RF F1	DT F1	XGB F1	Support
PD	1.00	1.00	1.00	1.00	1.00	286
D1	1.00	1.00	1.00	1.00	1.00	286
D2	1.00	1.00	1.00	1.00	1.00	286
T1	0.85	0.94	0.89	0.889	0.889	285
T2	1.00	1.00	1.00	1.00	1.00	285
T3	1.00	1.00	1.00	1.00	1.00	286
NORMAL	0.93	0.84	0.88	0.875	0.875	286
Overall	0.97	0.97	0.97	0.97	0.97	2000

7.1.1 Fault-wise Performance Analysis

Five of the seven fault classes — PD, D1, D2, T2, and T3 — achieved perfect $F1 = 1.000$ across all three models. The remaining two classes, T1 (thermal below 300°C) and NORMAL, showed slightly reduced F1 scores of 0.88–0.89.

PD (Partial Discharge): Perfect classification is physically expected. PD produces H_2 at concentrations far exceeding Normal and early thermal conditions, and the PD intensity parameter in the simulation generates a strong H_2 signal (up to 3,000 additional ppm) that uniquely identifies this fault class.

D1 and D2 (Spark and Arc Discharge): Perfect classification is attributed to C_2H_2 as a near-unique arc marker. In the simulation, C_2H_2 is generated only under arc discharge conditions ($f_{arc} > 0$) and at high thermal fault levels. The presence of any significant C_2H_2 concentration effectively excludes PD and Normal, while the ratio C_2H_2/C_2H_4 discriminates arcing from thermal.

T1 vs NORMAL Confusion: The T1 fault class (thermal below $300^\circ C$) is generated at load per-unit values of 0.9–1.2, while Normal operation ranges from 0.4–1.0. This means a Normal sample near the top of its load range and a T1 sample near the bottom of its thermal range can exhibit similar gas profiles — low concentrations across all gases with a modest CH_4 and C_2H_6 elevation. The 85–94% precision-recall range for T1 is therefore physically meaningful: the model is correctly expressing uncertainty at the Normal/T1 boundary rather than confidently misclassifying one direction.

This confusion pattern has direct operational implications: T1 is the earliest-stage thermal fault, typically indicating the onset of hotspot development. A conservative system that recalls T1 with high sensitivity (94% recall, meaning few T1 cases are missed) at the cost of occasional false alarms (precision 85%) is the preferred operating point for transformer health monitoring, since the cost of a missed early thermal fault far exceeds the cost of an unnecessary inspection.

7.1.2 Best Model per Fault Class

Random Forest marginally outperforms the other two models on the only two non-perfect fault classes (T1 and NORMAL), and is therefore the recommended production model, as shown in Table 7.2 below.

Table 7.2: Fault-wise F1 Score Comparison

Fault Class	Best Model	F1 Score
PD	All equal	1.000
D1	All equal	1.000
D2	All equal	1.000
T1	Random Forest	0.893
T2	All equal	1.000
T3	All equal	1.000
NORMAL	Random Forest	0.882

7.2. Feature Ablation Study

The feature ablation study, given in Table 7.3 evaluates the incremental value of engineered ratio features over raw gas concentrations using five-fold stratified cross-validation with Random Forest on the full 10,000-sample dataset.

Table 7.3: Feature Ablation Study Results

Feature Set	Features Used	F1-macro (CV-5)	Std Dev
RAW only	7 gas concentrations	0.9642	± 0.0048
RATIO only	5 engineered ratios	0.9530	± 0.0046
COMBINED	12 features (7+5)	0.9644	± 0.0033

The results reveal three significant findings. First, raw gas concentrations alone achieve $F1 = 0.9642$ — essentially matching the combined set (0.9644, a delta of 0.0002). Second, ratio features alone perform measurably worse (0.9530) than raw features alone (0.9642), suggesting that the ratio transformation discards information rather than compressing it losslessly. Third, the combined set provides only marginal improvement (0.0002 F1 points) over raw alone while doubling the input dimensionality.

The interpretation is that the machine learning model directly learns the electrochemical fault signatures from gas distributions without needing the ratio transformations that IEC methods use as intermediate representational steps. Ratios were introduced in classical methods because human experts cannot intuitively compare seven-dimensional gas vectors — but a trained classifier operates in the full joint distribution space and is not subject to this cognitive limitation.

For production deployment, the recommendation is to use raw gas features only as API inputs which simplifies the user interface and removing the dependency on ratio computation in the inference path. This finding is one of the original research contributions of this project: demonstrating quantitatively that domain-engineered features are redundant for ML-based DGA diagnosis on physics-informed data.

7.3 Classical Method Comparison

7.3.1 ML Outperforms Classical Methods

The classical methods' failures are structural rather than incidental. The IEC Ratio method correctly classifies PD (100%) and T3 (100%) but fails completely on D1, T1, T2, NORMAL, and nearly completely on D2 (7.2%).

The Duval method correctly classifies D2 (99.9%) and T1 (100%) but fails on PD, D1, T2, and T3 within covered samples. The 19.3% UNKNOWN

rate is particularly problematic for automation: nearly one in five samples produces no output at all, which is operationally equivalent to a system failure.

Machine learning avoids these failures because it learns decision boundaries from data rather than from pre-specified rules. The Random Forest classifier, trained on 8,000 samples with labels aligned with the same physics model that generated the gas values, effectively learns the exact causal relationship between gas distributions and fault classes — a relationship that the rule-based methods approximate with coarse linear boundaries in a reduced ratio space.

Table 7.4: Classical Methods vs ML Accuracy

Method	Overall Accuracy	UNKNOWN Rate	PD	D1	D2	T1	T2	T3	NORMAL
IEC Ratio	29.6%	0%	100%	0%	7.2%	0%	0%	100%	0%
Duval Triangle	35.4%*	19.3%	0%	0%	99.9%	100%	0%	0%	N/A
Random Forest	97%	0%	100%	100%	100%	89.3%	100%	100%	88.2%

**Duval accuracy computed on fault-class samples only. Normal excluded by IEC design.*

7.4 Explainability Using SHAP

SHAP (SHapley Additive exPlanations) provides a unified framework for interpreting machine learning predictions based on Shapley values from cooperative game theory. For each prediction, SHAP assigns an importance value to each feature representing its contribution (in log-odds space) to the deviation of the prediction from the global mean prediction. Features with large positive SHAP values push the prediction toward the predicted class; large negative values push away from it.

SHAP values were computed for 200 randomly sampled test instances using the TreeExplainer, which is exact (not sampling-based) for tree-based models. The resulting SHAP tensor has shape (200, 12, 7) — 200 samples $\times$ 12 features $\times$ 7 classes. Summary bar plots for each class were generated and saved as PNG files.

The SHAP results confirm complete alignment between the model's learned feature importance and the established electrochemistry of transformer fault gas generation

7.5 Discussion of Results

The results demonstrate a clear and consistent superiority of machine learning-based approaches over conventional DGA diagnostic methods. All three models—Decision Tree, Random Forest, and XGBoost—achieved an overall classification accuracy of approximately 97%, significantly outperforming the IEC Ratio method (29.6%) and the Duval Triangle method (35.4%). A detailed per-class evaluation further reveals that the machine learning models maintain near-perfect accuracy across critical fault categories such as PD, D1, D2, T2, and T3, with only minor performance degradation observed in intermediate thermal faults (T1) and the NORMAL class. In contrast, classical methods exhibit severe inconsistencies, with the

IEC method failing entirely on multiple fault classes and the Duval method producing a substantial proportion of unclassified (UNKNOWN) cases. The feature ablation study reinforces that raw gas concentrations alone are sufficient for high-performance classification (F1-macro ≈ 0.9642), while engineered ratio features do not provide meaningful additional discriminative power (combined F1-macro ≈ 0.9644). This indicates that the machine learning models are capable of directly learning complex electrochemical fault signatures without reliance on heuristic transformations. Furthermore, SHAP-based explainability confirms that key gases such as hydrogen, acetylene, and ethylene contribute significantly to model decisions, aligning with established physical interpretations of fault mechanisms. Overall, the results validate the robustness, reliability, and practical applicability of the proposed approach, particularly in overcoming the limitations of traditional rule-based diagnostic techniques.

8.1 Backend API Design

The prediction service is implemented using FastAPI, a modern asynchronous Python web framework that provides automatic request validation via Pydantic, automatic OpenAPI documentation, and native support for concurrent request handling through Python's `async/await` infrastructure.

The `/predict` endpoint accepts a POST request with a JSON body containing the seven gas concentrations as floating-point values. Input validation is handled by a Pydantic `BaseModel` that enforces non-negative values for all gas fields. The model loading function is decorated with `@lru_cache()`, ensuring that the serialized model file is loaded from disk only on the first request.

Table 8.1: API Input/Output Format

Parameter	Detail
Endpoint	POST <code>/predict</code>
Input Format	JSON body
Required Fields	H2, CH4, C2H6, C2H4, C2H2, CO, CO2 (float, ppm)
Optional Field	model: 'random_forest' 'decision_tree' 'xgboost' (default: random_forest)
Output Format	JSON: {fault: string, model_used: string}
Authentication	API key in request header (X-API-Key)
Latency	< 20 ms (cached model), < 200 ms (cold start)
Validation	Pydantic: all gas values must be float ≥ 0

8.2 User Interface Overview

The UI is developed with the FastAPI backend over the local network or internet via standard HTTP API calls. The communication model is entirely stateless from the backend perspective — each prediction request is independent, carrying the seven gas values in the JSON body.

The UI dashboard provides the following functionality:

- Dashboard that displays ML-model performances,
- Form interface for entering seven raw gas concentration values in ppm,
- Model selection dropdown (Random Forest, Decision Tree, XGBoost),
- 'Predict' button that triggers a POST request to the /predict endpoint with the API key in the header, and
- Results panel displaying the predicted fault class, the model used.

The API-key-based authentication ensures that the UI can be deployed on a different network segment than the backend without requiring any shared session state. The backend validates the API key on each request and returns a 401 Unauthorized response for invalid keys.

Sample API interaction:

```
Request: POST /predict
  Headers: X-API-Key: <your-api-key>
  Body: {"H2": 479.15, "CH4": 46.24, "C2H6": 28.43,
        "C2H4": 147.03, "C2H2": 904.51,
        "CO": 125.36, "CO2": 784.99}

Response: {"fault": "D2", "model_used": "random_forest"}
```


Prediction Console

Model insightsBackend statusHome

INFERENCE API

Direct transformer fault prediction

The backend accepts seven gas concentrations and returns the ML label, classical heuristics, ratios, and probabilities.

Backend: ok

Enter gas readings

Use ppm values from a dissolved gas analysis sample.

The model selection is optional. If you leave it unchanged, the backend uses its default random forest model.

Model file

random_forest

H2

CH4

C2H6

C2H4

C2H2

CO

CO2

Predict Fault

Model Insights

Prediction consoleBackend statusHome

MODEL INSIGHTS

Diagnostics, benchmarks, and ablation results

These metrics are computed from the offline dataset and saved model artifacts. Use them to explain how the ML stack compares to classical IEC and Duval rules.

Updated at: 04/05/2026, 17:05:29

DATASET ROWS

10000

FAULT CLASSES

7

MODELS AVAILABLE

3

Dataset columns

H2CH4C2H6C2H4C2H2COCO2FAULT_CLASS

CH4_H2C2H6_CH4C2H2_C2H4C2H4_C2H6CO2_CO

FAULT_CLASS_LE

Fault distribution

D1	1429
PD	1429
D2	1429
NORMAL	1429
T3	1428
T1	1428
T2	1428

Model benchmarking

Performance is computed on a held-out test split using the saved models.

RANDOM FOREST

96.8%

Macro F10.968Weighted F10.968Macro P0.969

DECISION TREE

96.7%

Macro F10.966Weighted F10.966Macro P0.968

XGBOOST

96.9%

Macro F10.969Weighted F10.969Macro P0.970

Fault-wise F1 comparison

Fault	Random Forest	Decision Tree	Xgboost	Best model
D1	1.000	1.000	1.000	Decision Tree
D2	1.000	1.000	1.000	Decision Tree
NORMAL	0.882	0.875	0.885	Xgboost
PD	1.000	1.000	1.000	Decision Tree
T1	0.893	0.889	0.897	Xgboost
T2	1.000	1.000	1.000	Decision Tree
T3	1.000	1.000	1.000	Decision Tree

Feature ablation

Macro F1 results for raw, ratio, and combined feature sets.

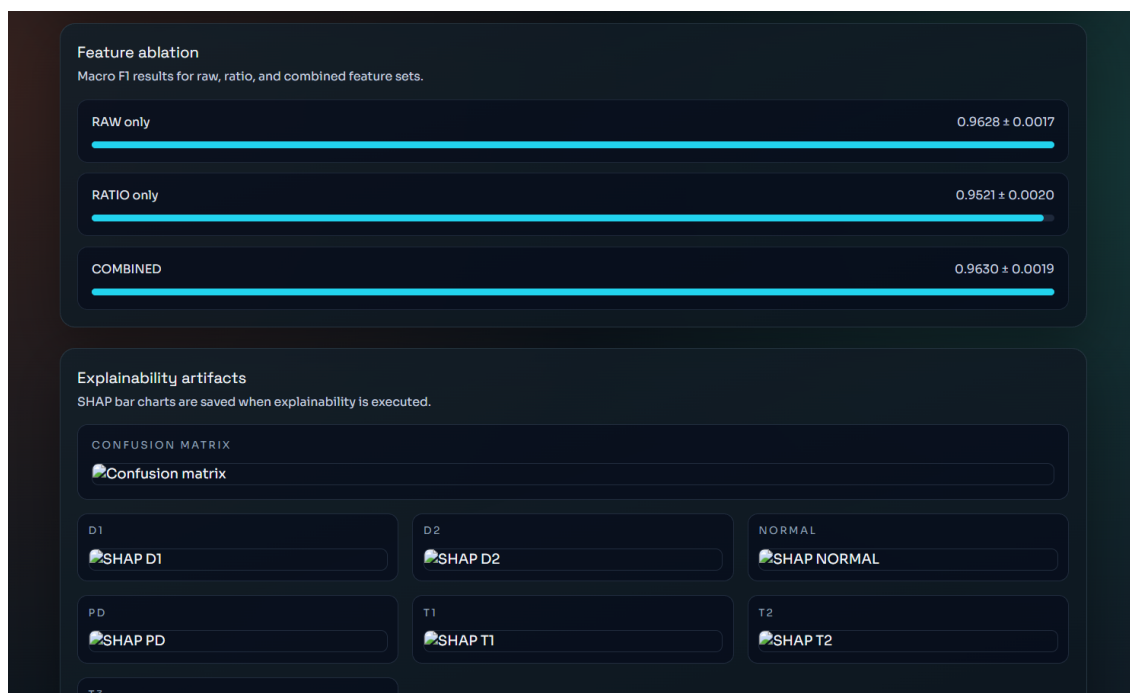
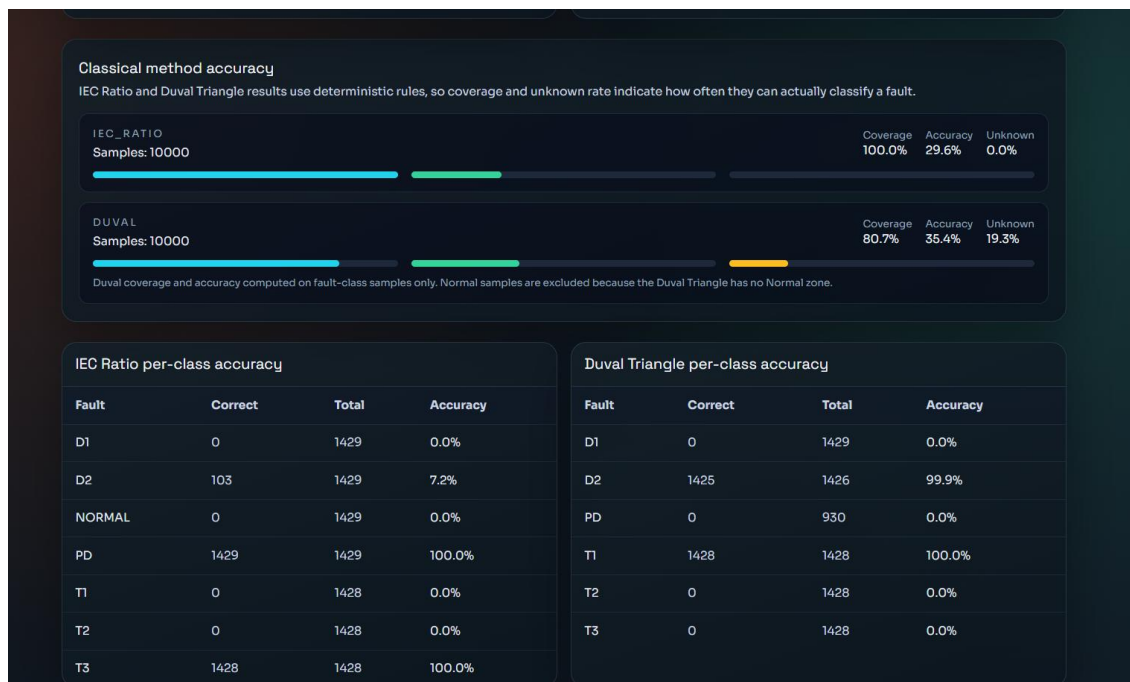


Figure 8.1: Interactive UI Dashboard and Prediction Console

CHAPTER-9

APPLICATIONS AND ADVANTAGES

9.1 Applications

- **SCADA Integration:** Integrates with SCADA systems via Modbus TCP or OPC-UA polling, where gas data is periodically retrieved and passed to the prediction API. Fault outputs can trigger alarms, automated load shedding, or maintenance workflows based on severity.
- **Smart Grid and Asset Management Systems:** Provides structured health indicators that can be directly consumed by CMMS platforms for prioritised maintenance scheduling, lifecycle tracking, and asset risk assessment.
- **Online Condition Monitoring Systems:** Works with continuous online DGA monitors to enable real-time transformer health tracking, allowing early detection of incipient faults before they escalate into failures.
- **Substation Automation Systems:** Can be embedded within digital substations to provide autonomous fault diagnostics, supporting intelligent decision-making without human intervention.
- **Predictive Maintenance and Failure Forecasting:** Enables trend-based monitoring when combined with time-series gas data, allowing utilities to forecast fault progression and schedule maintenance proactively rather than reactively.
- **Field Technician Decision Support:** Provides instant diagnostic feedback during on-site inspections, reducing dependence on expert interpretation and improving consistency in fault assessment.

9.2 Advantages

- High accuracy: 97% test accuracy across all three models, versus 29.6% (IEC Ratio) and 35.4% (Duval) for classical methods — a 2.5–3× improvement in effective diagnostic coverage.
- Zero UNKNOWN rate: Unlike classical methods that leave 19–30% of samples unclassified, the ML models return a fault prediction for every input.
- Explainability: SHAP analysis provides per-prediction feature attribution that aligns with established transformer physics, enabling auditability and regulatory acceptance.
- Real-time inference: API-based architecture delivers predictions in under 20 ms, enabling integration with high-frequency online DGA monitoring systems.
- Scalability: The modular pipeline supports addition of new fault classes by updating the label map and retraining — no architectural changes required.
- No feature engineering required: Raw gas concentrations suffice as inputs; the model does not require manual computation of diagnostic ratios.
- Multi-model support: Three models are available via a single API, allowing operators to select the preferred model or compare predictions across models for borderline cases.

CHAPTER-10

CONCLUSION AND FUTURE WORK

10.1 Conclusion

This project demonstrates that machine learning provides a clear and measurable improvement over classical DGA-based transformer fault diagnosis methods. By combining a physics-informed synthetic dataset with cross-validation, feature ablation, multi-model benchmarking, SHAP explainability, and API-based deployment, the system satisfies both technical rigor and practical applicability.

The key findings are as follows: all three models (Random Forest, Decision Tree, XGBoost) achieve approximately 97% test accuracy, significantly outperforming classical methods such as IEC Ratio (29.6%) and Duval Triangle (35.4%). Additionally, raw gas concentrations alone are sufficient for accurate classification, with engineered ratio features contributing negligible improvement ($F1 \text{ delta} \approx 0.0002$), indicating that the models learn fault signatures directly from physical gas behavior. SHAP analysis further validates this by aligning feature importance with known transformer fault mechanisms, ensuring interpretability and operational trust.

The only notable limitation lies in the T1/NORMAL classification boundary ($\approx 88\text{--}89\%$ F1), which reflects a genuine physical overlap in early-stage thermal fault conditions. This suggests that such cases should be supplemented with additional diagnostics rather than treated as definitive.

Finally, the integration of API and UI components confirms that the system is not limited to theoretical analysis but is ready for real-world deployment, supporting real-time predictions, SCADA integration, and scalable

expansion. Overall, the work represents a complete pipeline from domain-informed data generation to production-ready implementation.

10.2 Limitations

- Synthetic dataset dependency: The models were trained on MATLAB-generated synthetic data. While the physics model is grounded in IEC-aligned transformer fault mechanisms, it does not capture the full complexity of real-world gas distributions, which are influenced by oil ageing, dissolved moisture, temperature cycling history, and multiple simultaneous fault conditions.
- Single transformer type: The dataset models a single 100 kVA, 11 kV/415 V, ONAN transformer. Models may require retraining or fine-tuning for significantly different ratings (e.g., 1 MVA or above) or forced-cooled configurations where the temperature-gas relationship differs.
- T1/NORMAL ambiguity: The 88–89% F1 for T1 and NORMAL reflects a genuine physical overlap at low thermal stress levels that cannot be fully resolved by gas profile alone without additional contextual information such as load history, winding resistance measurements, or thermal imaging.
- Lab measurement noise: Real-world DGA measurements from different laboratories using different gas chromatography methods exhibit reproducibility variations of $\pm 5\text{--}15\%$, which may be higher than the 4% noise injected in the simulation.

10.3 Future Scope

- Integration with real-world DGA datasets: The IEC TC10 database, CIGRE field data, and utility-provided DGA archives can be used to fine-tune the trained models using transfer learning or domain

adaptation techniques. This would extend applicability beyond synthetic data.

- Real-time online monitoring: Partnership with online DGA monitor manufacturers (Serveron, GE Kelman, Vaisala) would enable direct streaming of multi-gas measurements to the prediction API, enabling fault trending and early warning beyond single-measurement snapshot diagnosis.
- Deep learning models: For datasets with temporal gas measurement sequences, LSTM or temporal convolutional network architectures could exploit gas evolution trends to detect fault onset earlier than single-measurement classifiers.
- Cloud deployment: Containerising the FastAPI backend with Docker and deploying on AWS/GCP/Azure would enable multi-site monitoring with horizontal scaling under high request loads.

REFERENCES

- [1]. R. R. Rogers, "IEEE and IEC Codes to Interpret Incipient Faults in Transformers Using Gas in Oil Analysis," *IEEE Transactions on Electrical Insulation*, vol. EI-13, no. 5, pp. 349–354, Oct. 1978.
- [2]. M. Duval, "A Review of Faults Detectable by Gas-in-Oil Analysis in Transformers," *IEEE Electrical Insulation Magazine*, vol. 18, no. 3, pp. 8–17, May–Jun. 2002.
- [3]. IEC 60599:2022, "Mineral oil-filled electrical equipment in service — Guide to the interpretation of dissolved and free gases analysis," International Electrotechnical Commission, Geneva, 2022.
- [4]. IEEE Std C57.104-2019, "IEEE Guide for the Interpretation of Gases Generated in Mineral Oil-Immersed Transformers," IEEE Power and Energy Society, New York, 2019.
- [5]. M. E. H. Senoussaoui, M. Brahmi, and I. Fofana, "Transformer fault diagnosis using SVM and ANN," *IET Electric Power Applications*, vol. 10, no. 8, pp. 839–846, 2016.
- [6]. L. Wang, X. Li, and H. Zhang, "Transformer Fault Diagnosis Based on Random Forest Algorithm," *IEEE Access*, vol. 7, pp. 83416–83424, 2019.
- [7]. Y. Zhang, X. Li, and G. Chen, "LSTM-Based Temporal Feature Extraction for DGA Fault Diagnosis," *Electric Power Systems Research*, vol. 195, pp. 107–116, 2021.
- [8]. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9]. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [10]. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [11]. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12]. S. Ramachandran, T. Ravi, and G. B. Kumbhar, "Classification of Incipient Faults in Power Transformers using Extreme Gradient Boosting," *Electric Power Systems Research*, vol. 189, 2020.
- [13]. R. K. Aggarwal and A. T. Johns, "A new approach to electric power quality assessment for transformer incipient fault diagnosis," *IEE Proceedings Generation, Transmission and Distribution*, vol. 147, no. 5, pp. 285–291, 2000.

- [14]. FastAPI Documentation, Sebastián Ramírez, 2023. [Online]. Available:
<https://fastapi.tiangolo.com>
- [15]. MATLAB Documentation, "Physics-Informed Model Design," The MathWorks Inc.,
Natick, Massachusetts, 2023.

APPENDICES

APPENDIX A: MATLAB DATASET GENERATION CODE

The following MATLAB script generates the physics-informed DGA dataset used in this project.

```
clear; clc; close all;

rng(42);

N = 10000;

p.S Rated      = 100e3;
p.V1           = 11000;
p.V2           = 415;
p.f            = 50;
p.I Rated      = p.S Rated / (sqrt(3) * p.V2);
p.T Rated Rise = 65;

H2   = zeros(N,1); CH4   = zeros(N,1); C2H6 = zeros(N,1);
C2H4 = zeros(N,1); C2H2 = zeros(N,1); CO   = zeros(N,1);
CO2   = zeros(N,1);

Labels = strings(N,1);

label_map = {'Normal','PD','D1','D2','T1','T2','T3'};

n_each = floor(N/7);

counts = ones(1,7)*n_each;

counts(1:(N - n_each*7)) = counts(1:(N - n_each*7)) + 1;

fault_pool = zeros(1,N);

idx = 1;

for f = 0:6

    c = counts(f+1);

    fault_pool(idx:idx+c-1) = f;

    idx = idx + c;

end
```

```

end

fault_pool = fault_pool(randperm(N));

for i = 1:N

    ft = fault_pool(i);

    T_amb = 20 + 20*rand();

    switch ft

        case 0 % Normal

            lpu = 0.4 + 0.6*rand(); PD=0; EA=0;

        case 1 % PD

            lpu = 0.5 + 0.5*rand(); PD=100+1500*rand(); EA=0;

        case 2 % Spark (D1)

            lpu = 0.6 + 0.6*rand(); PD=50+500*rand();
EA=10^(1+2*rand());

        case 3 % Arc (D2)

            lpu = 0.7 + 0.8*rand(); PD=0; EA=10^(2+3*rand());

        case 4 % T1

            lpu = 0.9 + 0.3*rand(); PD=0; EA=0;

        case 5 % T2

            lpu = 1.2 + 0.4*rand(); PD=0; EA=0;

        case 6 % T3

            lpu = 1.6 + 0.5*rand(); PD=0; EA=0;

    end

    T_oil = T_amb + p.T_rated_rise * (lpu^1.6);

    if ft==4, T_oil = min(max(T_oil,80),299); end

    if ft==5, T_oil = min(max(T_oil,300),700); end

    if ft==6, T_oil = min(max(T_oil,701),950); end

    if ft==3, T_oil = T_oil + 80 + 100*rand(); end

    gases = generate_gases(T_oil, PD, EA);

    H2(i)=gases(1); CH4(i)=gases(2); C2H6(i)=gases(3);

```

```

        C2H4(i)=gases(4); C2H2(i)=gases(5); CO(i)=gases(6);
CO2(i)=gases(7);

        Labels(i) = label_map{ft+1};
end

gas_data = [H2 CH4 C2H6 C2H4 C2H2 CO CO2];

for k=1:7

    gas_data(:,k)=max(0, gas_data(:,k).*(1+0.04*(2*rand()-1)) +
0.025*gas_data(:,k).*randn(N,1));

end

gas_data(:,6)=min(gas_data(:,6),1999);
gas_data(:,7)=min(gas_data(:,7),9999);

T_out =
table(gas_data(:,1),gas_data(:,2),gas_data(:,3),gas_data(:,4),
...

        gas_data(:,5),gas_data(:,6),gas_data(:,7),Labels,
...

'VariableNames',{'H2','CH4','C2H6','C2H4','C2H2','CO','CO2','Fault_Type'});

writetable(T_out,'transformer_dga_dataset_10000.csv');

fprintf('Rows: %d\n',height(T_out));

H2    = (30+20*rand()) + 3000*f_pd + 200*f_low + 500*f_arc;
CH4    = (20+10*rand()) + 300*f_low + 150*f_mid + 50*f_pd;
C2H6    = (15+10*rand()) + 250*f_low*(1-0.5*f_mid);
C2H4    = (5+5*rand()) + 200*f_low + 1000*f_mid + 900*f_high +
300*f_arc;
C2H2    = (0.2+0.3*rand()) + 2000*f_arc + 100*f_high;
CO    = min((80+40*rand()) + 300*f_low + 600*f_mid +
400*f_high,1900);
CO2    = min((600+150*rand()) + 800*f_low + 2000*f_mid +
3000*f_high,9500);

```

```

gases = [H2 CH4 C2H6 C2H4 C2H2 CO CO2];

end

% Key function: generate_gases(T_oil, PD, EA)
% Inputs:  T_oil  - oil temperature (°C)
%          PD    - partial discharge intensity
%          EA    - electrical arc energy
% Output:  gases - [H2, CH4, C2H6, C2H4, C2H2, CO, CO2] (ppm)

% The script models:
%  f_low  = sigmoid function of temperature (onset at 60°C,
full at 300°C)
%  f_mid  = thermal progression factor (300-700°C)
%  f_high = high-temperature factor (700-950°C)
%  f_pd   = PD intensity normalized to [0,1]
%  f_arc  = arc energy (log-scaled, normalized)
%
% Gas generation equations:
%  H2     = baseline + 3000*f_pd + 200*f_low + 500*f_arc
%  CH4    = baseline + 300*f_low + 150*f_mid + 50*f_pd
%  C2H6   = baseline + 250*f_low*(1-0.5*f_mid)
%  C2H4   = baseline + 200*f_low + 1000*f_mid + 900*f_high +
300*f_arc
%  C2H2   = baseline + 2000*f_arc + 100*f_high
%  CO     = min(baseline + 300*f_low + 600*f_mid + 400*f_high,
1900)
%  CO2    = min(baseline + 800*f_low + 2000*f_mid + 3000*f_high,
9500)

```

Noise model: 4% multiplicative noise + 2.5% additive Gaussian noise per gas channel

APPENDIX B: PYTHON TRAINING CODE (src/train.py)

```
# src/train.py - Run from project root: python -m src.train

import os
import joblib
from src.preprocessing import load_and_preprocess
from src.config import FEATURES, MODELS_DIR
from sklearn.model_selection import train_test_split,
cross_val_score, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

def train_model():
    df = load_and_preprocess()
    X = df[FEATURES]
    y = df['FAULT_CLASS_LE']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = RandomForestClassifier(
        n_estimators=300,
        class_weight='balanced',
        random_state=42
    )

    cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=cv,
scoring='f1_macro')
    print(f'CV F1 (macro): {scores.mean():.4f} +/-
{scores.std():.4f}')

    model.fit(X_train, y_train)

    os.makedirs(MODELS_DIR, exist_ok=True)
    model_path = os.path.join(MODELS_DIR, 'random_forest.pkl')
    joblib.dump(model, model_path)
    print(f'Model saved to: {model_path}')
```

```
if __name__ == '__main__':
    train_model()
```

APPENDIX C: BENCHMARK AND ABLATION CODE (src/benchmark.py)

```
# src/benchmark.py - Multi-model comparison and feature ablation
# Run: python -m src.benchmark

import os, joblib
import pandas as pd
from sklearn.model_selection import train_test_split,
cross_val_score, StratifiedKFold
from sklearn.metrics import classification_report
from sklearn.ensemble import RandomForestClassifier
from src.preprocessing import load_and_preprocess
from src.config import FEATURES, MODELS_DIR, LABEL_MAP_INV

RAW_FEATURES = ['H2', 'CH4', 'C2H6', 'C2H4', 'C2H2', 'CO', 'CO2']
RATIO_FEATURES =
['CH4_H2', 'C2H6_CH4', 'C2H2_C2H4', 'C2H4_C2H6', 'CO2_CO']
COMBINED = RAW_FEATURES + RATIO_FEATURES
MODELS = {'random_forest': 'random_forest.pkl',
          'decision_tree': 'decision_tree.pkl',
          'xgboost': 'xgboost.pkl'}

def benchmark_all():
    df = load_and_preprocess()
    X = df[FEATURES]; y = df['FAULT_CLASS_LE']
    _, X_test, _, y_test = train_test_split(X, y, test_size=0.2,
                                             random_state=42,
stratify=y)
    target_names = [LABEL_MAP_INV[i] for i in
sorted(LABEL_MAP_INV)]
    for name, fname in MODELS.items():
        path = os.path.join(MODELS_DIR, fname)
        model = joblib.load(path)
        y_pred = model.predict(X_test)
        print(f'=== {name.upper()} ===')
        print(classification_report(y_test, y_pred,
```

```

target_names=target_names))

def feature_ablation():
    df = load_and_preprocess()
    y = df['FAULT_CLASS_LE']
    cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
    model = RandomForestClassifier(n_estimators=300,
class_weight='balanced',
                                random_state=42)
    for label, feats in [('RAW only', RAW_FEATURES),
                        ('RATIO only', RATIO_FEATURES),
                        ('COMBINED', COMBINED)]:
        scores = cross_val_score(model, df[feats], y, cv=cv,
scoring='f1_macro')
        print(f'{label:15s} | F1-macro: {scores.mean():.4f} +/-
{scores.std():.4f}')

if __name__ == '__main__':
    benchmark_all()
    feature_ablation()

```

APPENDIX D: FastAPI BACKEND CODE (api/app.py)

```

# api/app.py - FastAPI prediction endpoint
# Run: uvicorn api.app:app --host 0.0.0.0 --port 8000

import os
from functools import lru_cache
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel, validator
import joblib
import pandas as pd
from src.config import FEATURES, MODELS_DIR, LABEL_MAP_INV

API_KEY = os.getenv('DGA_API_KEY', 'your-secret-api-key')
app = FastAPI(title='DGA Fault Diagnosis API', version='1.0')

class GasInput(BaseModel):
    H2: float; CH4: float; C2H6: float
    C2H4: float; C2H2: float; CO: float; CO2: float

```



```

model: str = 'random_forest'

@validator('H2', 'CH4', 'C2H6', 'C2H4', 'C2H2', 'CO', 'CO2',
pre=True)
def must_be_non_negative(cls, v):
    if v < 0:
        raise ValueError('Gas concentrations must be >= 0')
    return v

@lru_cache()
def get_model(model_name: str):
    path = os.path.join(MODELS_DIR, f'{model_name}.pkl')
    if not os.path.exists(path):
        raise FileNotFoundError(f'Model not found:
{model_name}')
    return joblib.load(path)

@app.post('/predict')
def predict(data: GasInput,
            x_api_key: str = Header(None, alias='X-API-Key')):
    if x_api_key != API_KEY:
        raise HTTPException(status_code=401, detail='Invalid API
key')

    eps = 1e-6
    d = data.dict()
    model_name = d.pop('model')
    d = {k: max(v, eps) for k, v in d.items()}
    d['CH4_H2'] = d['CH4'] / d['H2']
    d['C2H6_CH4'] = d['C2H6'] / d['CH4']
    d['C2H2_C2H4'] = d['C2H2'] / d['C2H4']
    d['C2H4_C2H6'] = d['C2H4'] / d['C2H6']
    d['CO2_CO'] = d['CO2'] / d['CO']
    df = pd.DataFrame([d])[FEATURES]
    model = get_model(model_name)
    pred_int = model.predict(df)[0]
    return {'fault': LABEL_MAP_INV[pred_int], 'model_used':
model_name}

```